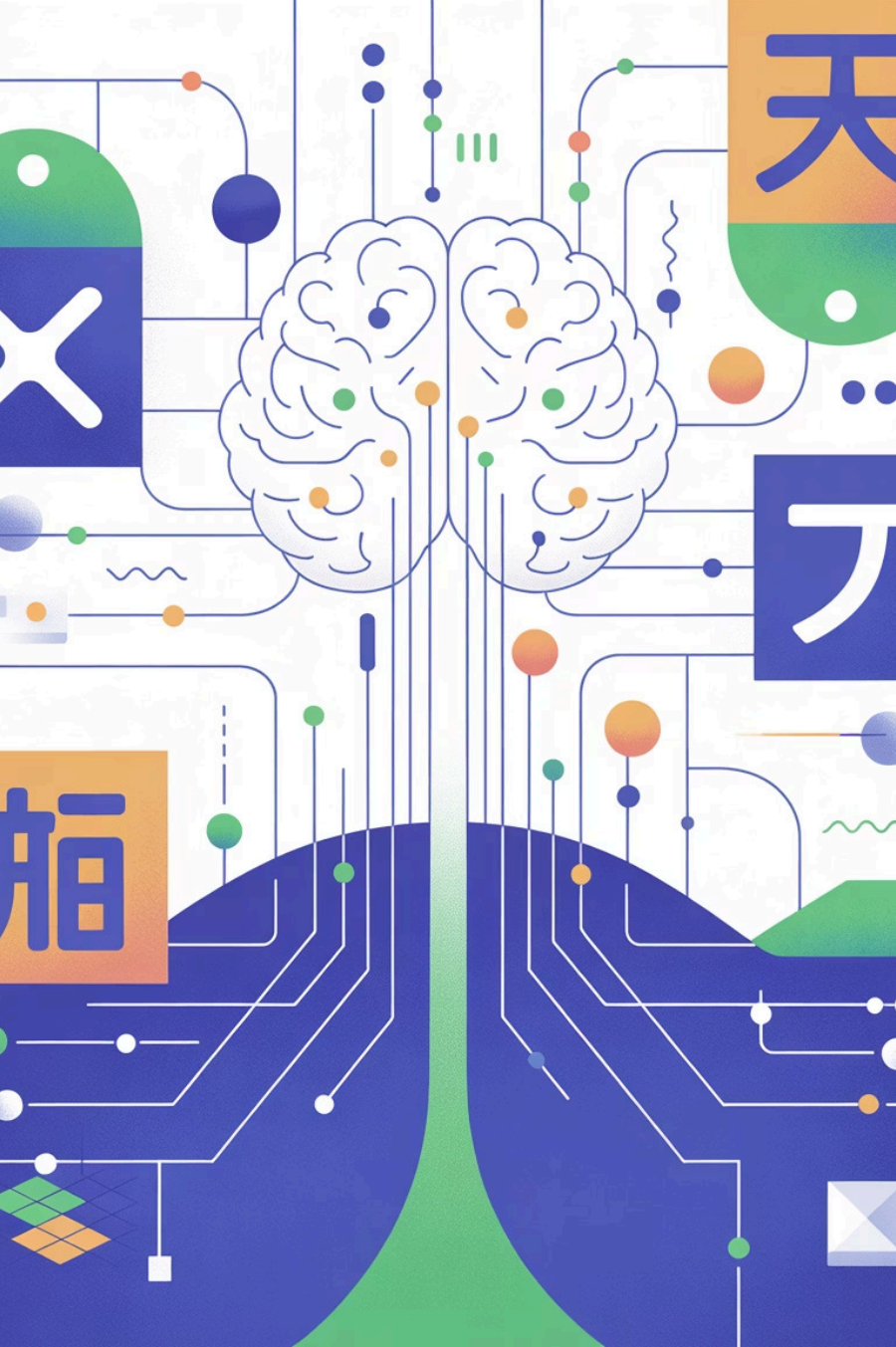


Natural Language Processing

Transforming Human Language into Machine Intelligence





What is NLP?



Machine Understanding

NLP enables computers to understand, interpret, and generate human language in meaningful ways.



Text to Numbers

The core challenge involves converting unstructured text into numerical representations that machines can analyze.



Real-World Applications

Powers chatbots, sentiment analysis, spam detection, translation services, and voice assistants.

Text Processing



Text Preprocessing: The Foundation

Key Preprocessing Steps

Tokenization breaks text into manageable units—sentences or individual words—creating the building blocks for analysis.

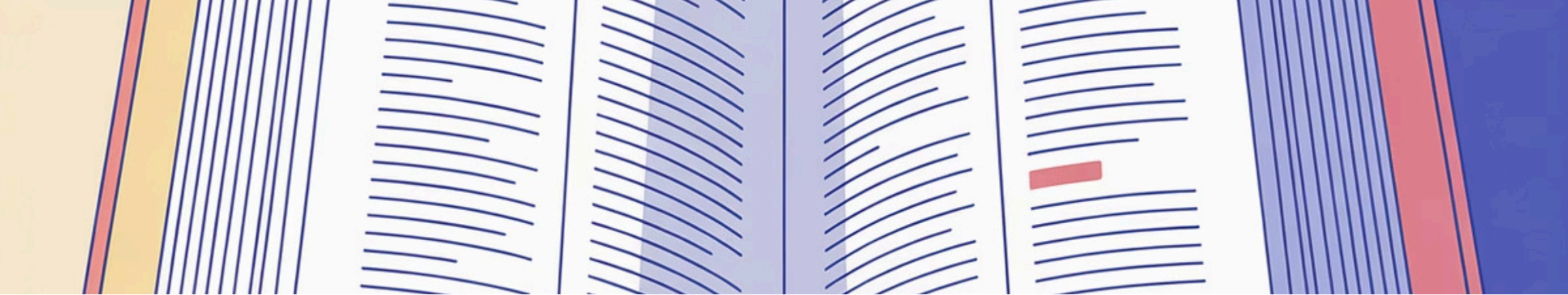
Stopwords Removal filters out common words like "the", "is", and "and" that carry minimal semantic value but create noise in the data.

Example Transformation

❏ **Original:**
"Fake news is misleading."



❏ **After Processing:**
["fake", "news", "misleading"]



Lemmatization vs Stemming



Stemming

Crude chopping of word endings to find the root. Fast but imprecise—may produce non-words.

Example: "arguing" → "argu"



Lemmatization

Uses vocabulary and morphological analysis to return the meaningful dictionary form of words.

Example: "arguing" → "argue"

 **Best Practice:** Lemmatization is preferred for accuracy-critical NLP tasks, though it requires more computational resources than stemming.



Bag of Words (BoW) Model

The Bag of Words approach creates a vocabulary of all unique words in your corpus, then represents each document as a vector counting word occurrences.

01

Build Vocabulary

Extract all unique words from the corpus to create your feature set.

02

Count Occurrences

For each document, count how many times each vocabulary word appears.

03

Create Vectors

Transform documents into numerical vectors for machine learning algorithms.

Example Corpus

1. "Fake news is fake."
2. "Audience encourages fake news."

Vocabulary:

["fake", "news", "audience", "encourages"]

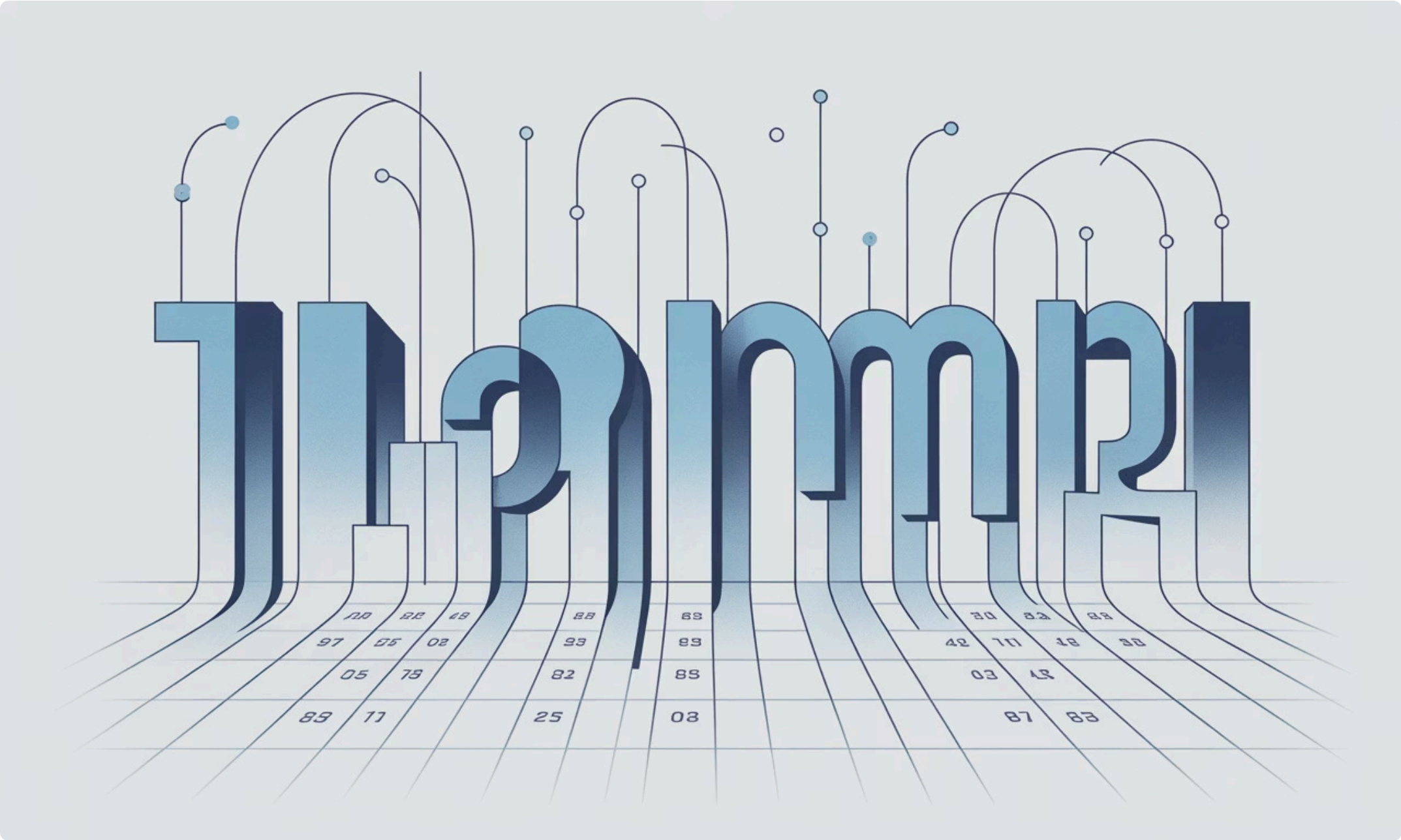
Document Vectors

❏ **Document 1:** [2, 1, 0, 0]
"fake" appears twice, "news" once

❏ **Document 2:** [1, 1, 1, 1]
Each word appears once

Understanding Features in Text Vectorization

In Natural Language Processing (NLP), "features" are the fundamental building blocks that represent text data in a numerical format digestible by machine learning models.



Specifically:

- Features are the individual dimensions or columns in your vector representation of a document.
- Each unique word in the vocabulary typically becomes one feature.
- The values within these features are the numerical scores, such as TF-IDF scores (or simple word counts, as seen in the Bag of Words model).

Example: Sentiment Analysis Features

Let's consider a simple sentiment analysis task. We want to train a model to classify text as "Positive" or "Negative" based on its content.

Sample Sentiment Data

Text	Sentiment
"This movie is great"	Positive
"I love this movie"	Positive
"This movie is terrible"	Negative
"I hate this movie"	Negative

Features (Vocabulary)

From our sample data, we extract a vocabulary of unique words, which will become our features:

["this", "movie", "is", "great", "love", "terrible", "hate"]

TF-IDF Vectorization Table

Each text sample is then transformed into a vector where each column corresponds to a feature, and the values are their TF-IDF scores:

Text	this	movie	is	great	love	terrible	hate	Sentiment
"This movie is great"	0.5	0.5	0.5	0.7	0	0	0	Positive
"I love this movie"	0.5	0.5	0	0	0.7	0	0	Positive
"This movie is terrible"	0.5	0.5	0.5	0	0	0.7	0	Negative
"I hate this movie"	0.5	0.5	0	0	0	0	0.7	Negative

Interpreting the Vectorization

- ☐ **Feature Representation:** Each column (except for "Sentiment") in the table above represents a distinct FEATURE extracted from our text data.
- ☐ **Document Vectors:** Each row corresponds to a single document or text sample, now represented as a numerical vector.
- ☐ **TF-IDF Scores:** The numbers within the table are the TF-IDF scores, indicating the importance of each word within a specific document relative to the entire corpus.
- ☐ **ML Input:** This tabular format is exactly what machine learning algorithms receive as input. The algorithms then process these numerical vectors.
- ☐ **Learning Patterns:** The ML model learns patterns from these features; for instance, it might learn that high TF-IDF scores for "great" or "love" often correlate with a "Positive" sentiment, while "terrible" or "hate" correlate with "Negative" sentiment.

Word2Vec: Capturing Word Meaning

Word2Vec is a revolutionary technique that represents words as dense vectors in a continuous vector space, where semantically similar words are positioned close to each other. Unlike TF-IDF which creates sparse vectors based on word frequency, Word2Vec learns word meanings from context.

Key Concepts

Dense Embeddings

- Word2Vec creates dense vectors (typically 100-300 dimensions) instead of sparse vocabulary-sized vectors
- Each dimension captures latent semantic features
- Example: "king" → [0.2, -0.5, 0.8, ..., 0.3]

Context-Based Learning

- Learns from word co-occurrence patterns in large text corpora
- Two architectures: CBOW (predicts word from context) and Skip-gram (predicts context from word)
- Captures semantic relationships

Semantic Relationships

- Famous example: king - man + woman ≈ queen
- Similar words have similar vectors: "happy" is close to "joyful", "pleased"
- Captures analogies and word relationships

Why Word2Vec is Important

- Captures semantic meaning, not just frequency
- Handles synonyms effectively (TF-IDF treats "happy" and "joyful" as completely different)
- Reduces dimensionality (300 dimensions vs 10,000+ vocabulary size)
- Pre-trained models available (trained on billions of words)
- Foundation for modern NLP (BERT, GPT build on these concepts)

Combining TF-IDF and Word2Vec

To leverage the strengths of both TF-IDF's frequency importance and Word2Vec's semantic understanding, various combination approaches can be used.

Approach 1: Weighted Word2Vec

- Calculate TF-IDF scores for words in a document
- Get Word2Vec embeddings for each word
- Create document vector by taking weighted average: `sum(TF-IDF_score × Word2Vec_embedding)` for all words
- Result: Document representation that combines frequency importance with semantic meaning

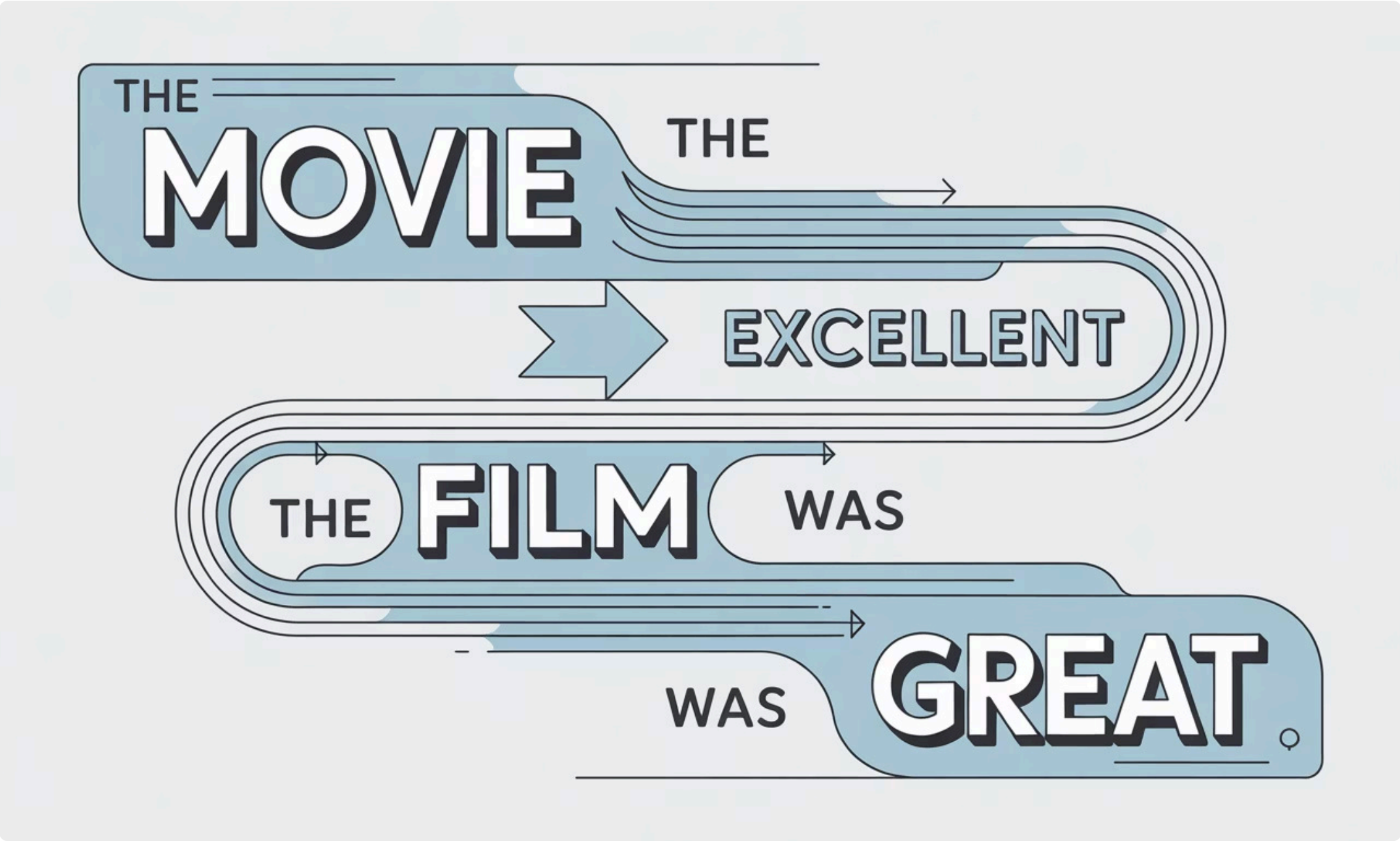
Approach 2: Hybrid Features

- Use TF-IDF vectors as one set of features
- Use averaged Word2Vec embeddings as another set
- Concatenate both feature sets
- Feed combined features to ML model
- Best of both worlds: frequency statistics + semantic understanding

Example showing the difference:

Documents: "The movie was excellent" vs "The film was great"

- TF-IDF:** Treats these as completely different (no shared words except "the", "was")
- Word2Vec:** Recognizes similarity because "movie"≈"film" and "excellent"≈"great"
- Combined:** Captures both frequency patterns AND semantic similarity



BoW vs TF-IDF: A Visual Comparison

Bag of Words Vector

Example: [2, 1, 0, 0]

Simple frequency counts—every occurrence weighted equally.

TF-IDF Vector

Example: [0.5, 0.3, 0, 0]

Weighted scores—reduces common words, boosts rare ones.

The Impact

TF-IDF helps machine learning models focus on the most informative terms for tasks like document classification, information retrieval, and content recommendations. Words that appear everywhere become less influential, while distinctive vocabulary receives proper emphasis.

[illegible]

Example: "Dogs/NOUN run/VERB quickly/ADVERB"

- Bigrams (2 words): "New York"
- Trigrams (3 words): "natural language processing"

Example: "Apple/ORG released iPhone/PRODUCT in California/LOC"

Common examples: "the", "is", "at", "which", "on"



Interactive Exercise

Build Your Own BoW Vector

Try It Yourself

Enter a short sentence in the form below to see NLP preprocessing in action:

1. Tokenization breaks your text apart
2. Stopwords get filtered out
3. Lemmatization normalizes words
4. BoW vector generated from vocabulary



Input Form

Enter your sentence:

[Text input field]

Processing options:



Remove stopwords



Apply lemmatization



Show TF-IDF weights

[Process & Show Results Button]

Expected Output Display: Your processed tokens, vocabulary list, and resulting BoW vector will appear below after processing.

Key Takeaways & Next Steps

Core Concept

NLP transforms unstructured text into numerical representations through preprocessing techniques like tokenization, lemmatization, and vectorization.

Foundational Methods

Bag of Words and TF-IDF serve as essential feature extraction techniques that convert text into machine-readable formats.

Best Practices

Lemmatization provides more accurate text normalization than stemming by preserving meaningful word forms.

NEXT

Advanced Topics to Explore

- Word2Vec and GloVe embeddings
- Transformer models (BERT, GPT)
- Attention mechanisms
- Sequence-to-sequence models

Hands-On Practice

- Work with real-world datasets
- Build sentiment classifiers
- Experiment with different vectorization techniques
- Compare model performance

Ready to Master AI & NLP?

This presentation covered the fundamentals of Natural Language Processing, but there's so much more to explore in the world of Artificial Intelligence and Machine Learning.



Enroll in our comprehensive AI course to dive deeper into:

- Advanced NLP techniques and transformer models
- Deep Learning and Neural Networks
- Computer Vision and Image Processing
- Real-world AI applications and projects
- Hands-on coding experience with industry tools